

CS121 Lab 2: GPU Cuckoo Hashing

1. Algorithm Description

1.1 Cuckoo Hashing Overview

Cuckoo hashing is an open-addressing hash table scheme that achieves $O(1)$ worst-case lookup time. Each key can be placed in one of k possible locations determined by k independent hash functions. When inserting a key that finds all its locations occupied, it "evicts" an existing key and relocates it to one of its alternative positions, creating an eviction chain.

1.2 GPU Implementation

My implementation parallelizes cuckoo hashing on NVIDIA GPUs using CUDA:

Data Structures:

- `d_table` : GPU array storing the hash table (uint32_t values)
- `d_seeds` : Three hash function seeds stored in GPU memory
- `EMPTY_KEY = 0xFFFFFFFF` : Sentinel value for empty slots

Hash Functions: We use three independent hash functions to minimize correlation:

- MurmurHash3** - High-quality general-purpose hash
- xxHash32** - Fast hash with good distribution
- FNV-1a** - Simple but effective hash

Insert Algorithm (per thread):

- ```
1. Check if key already exists in any of its k positions
2. For up to maxEvict iterations:
 a. Compute hash location for current hash index
 b. If slot is empty, use atomicCAS to insert
 c. If occupied, use atomicExch to evict existing key
 d. If evicted, the evicted key becomes the new key to insert
 e. Rotate to next hash function
3. If maxEvict exceeded, set global failure flag
```

**Concurrency Handling:**

- `atomicCAS` for empty slot insertion (prevents lost updates)
- `atomicExch` for eviction (atomic swap)
- Different threads start from different hash indices ( `idx % numHashes` ) to reduce contention
- Random state seeded with thread index + clock for symmetry breaking

**Rehashing:** When insertion fails (eviction chain too long), we:

- Generate new random seeds for all hash functions
- Clear the table
- Retry insertion with new hash functions

### 1.3 Lookup Algorithm

Lookup is straightforward and highly parallel:

```
For each hash function h in [0, numHashes):
 if table[hash_h(key)] == key:
 return FOUND
return NOT_FOUND
```

## 2. Machine Configuration

**GPU:** NVIDIA Tesla V100-PCIE-32GB

| Specification         | Value        |
|-----------------------|--------------|
| Compute Capability    | 7.0 (SM 7.0) |
| Global Memory         | 32 GB HBM2   |
| Multiprocessors (SMs) | 80           |

| Specification     | Value       |
|-------------------|-------------|
| CUDA Cores        | 5120        |
| Max Threads/Block | 1024        |
| Memory Bandwidth  | 900 GB/s    |
| FP32 Performance  | 15.7 TFLOPS |

### 3. Implementation Details

#### 3.1 Key Parameters

| Parameter    | Value                  | Description                  |
|--------------|------------------------|------------------------------|
| BLOCK_SIZE   | 256                    | Threads per CUDA block       |
| EMPTY_KEY    | 0xFFFFFFFF             | Sentinel for empty slots     |
| MAX_RESTARTS | 100                    | Maximum rehash attempts      |
| maxEvict     | 4×log <sub>2</sub> (n) | Default eviction chain limit |

#### 3.2 Code Structure

```
src/
├─ main.cu # Entry point, argument parsing
├─ common.cu/cuh # Utilities, timing, key generation
├─ hashtable.cu/cuh # Hash table class and CUDA kernels
├─ benchmark.cu/cuh # Benchmarking functions
├─ experiments.cu/cuh # Experiment implementations
└─ fileio.cu/cuh # File-based operations
```

### 4. Optimizations and Design Decisions

#### 4.1 Implemented Optimizations

- Early Empty Check:** Before using `atomicCAS`, we first read the slot with a normal load. This avoids expensive atomic operations when the slot is clearly occupied.
- Staggered Hash Start:** Each thread starts from hash index `idx % numHashes`, distributing initial insertions across different hash functions and reducing contention.
- Duplicate Detection:** Before attempting insertion, we check all `k` positions to see if the key already exists, avoiding unnecessary work.
- Random Symmetry Breaking:** When concurrent operations cause unexpected states, we use a per-thread random number generator to select the next hash function, preventing deadlocks.

#### 5.2 Problems Encountered

- Eviction Chain Cycles:** In parallel execution, multiple threads can create circular eviction chains. We handle this by limiting `maxEvict` and triggering rehash on failure.
- Load Factor Sensitivity:** At high load factors ( $< 1.2 \times$  table size), insertion success rate drops significantly, requiring more rehash attempts.
- Memory Coalescing:** Random hash access patterns inherently have poor memory coalescing. This is a fundamental limitation of hash table designs.

### 5. Compilation and Execution

#### 5.1 Build

```
make clean && make
```

Or use the provided script:

```
./run_experiments.sh
```

## 5.2 Run Experiments

```
Run all experiments with 2 and 3 hash functions
./cuckoo_hash --all

Run specific experiment
./cuckoo_hash --exp1 --hash2 # Experiment 1 with 2 hash functions
./cuckoo_hash --exp2 --hash3 # Experiment 2 with 3 hash functions
./cuckoo_hash --exp3
./cuckoo_hash --exp4
```

## 5.3 File-based Operations

Create a hash table from keys in a file and query with another file:

```
./cuckoo_hash --file <insert_keys> <query_keys> <output> [--hash2|--hash3]
```

**File Format:**

- Input files: One unsigned integer key per line
- Output file: One result per line (1 = found, 0 = not found)

**Example:**

```
Create test data
seq 1 10000 > insert.txt
seq 5000 15000 > query.txt

Build hash table and query
./cuckoo_hash --file insert.txt query.txt results.txt --hash2

Check results
cat results.txt
```

## 6. Results Summary

### Experiment 1: Insertion Throughput

**2 Hash Functions:**

| N  | Keys      | M ops/sec | Status |
|----|-----------|-----------|--------|
| 10 | 1,024     | 3.67      | OK     |
| 11 | 2,048     | 7.54      | OK     |
| 12 | 4,096     | 15.04     | OK     |
| 13 | 8,192     | 29.76     | OK     |
| 14 | 16,384    | 58.60     | OK     |
| 15 | 32,768    | 111.40    | OK     |
| 16 | 65,536    | 197.71    | OK     |
| 17 | 131,072   | 332.15    | OK     |
| 18 | 262,144   | 537.04    | OK     |
| 19 | 524,288   | 755.26    | OK     |
| 20 | 1,048,576 | 976.11    | OK     |

| N  | Keys       | M ops/sec | Status |
|----|------------|-----------|--------|
| 21 | 2,097,152  | 1,100.95  | OK     |
| 22 | 4,194,304  | 1,159.81  | OK     |
| 23 | 8,388,608  | 1,166.47  | OK     |
| 24 | 16,777,216 | 0.00      | FAILED |

3 Hash Functions:

| N  | Keys       | M ops/sec | Status |
|----|------------|-----------|--------|
| 10 | 1,024      | 4.14      | OK     |
| 11 | 2,048      | 8.32      | OK     |
| 12 | 4,096      | 16.53     | OK     |
| 13 | 8,192      | 32.54     | OK     |
| 14 | 16,384     | 63.20     | OK     |
| 15 | 32,768     | 117.82    | OK     |
| 16 | 65,536     | 205.47    | OK     |
| 17 | 131,072    | 331.00    | OK     |
| 18 | 262,144    | 490.83    | OK     |
| 19 | 524,288    | 658.19    | OK     |
| 20 | 1,048,576  | 807.49    | OK     |
| 21 | 2,097,152  | 881.42    | OK     |
| 22 | 4,194,304  | 916.41    | OK     |
| 23 | 8,388,608  | 918.91    | OK     |
| 24 | 16,777,216 | 872.09    | OK     |

**Key Observation:** 2 hash functions fail at 2^24 keys (50% load factor), while 3 hash functions succeed with 872 M ops/sec.

### Experiment 2: Lookup Performance (3 Hash Functions)

*Note: 2 hash functions failed to build the table, so only 3 hash function results are available.*

| Hit Rate | M ops/sec |
|----------|-----------|
| 0%       | 6,595.97  |
| 10%      | 5,972.37  |
| 20%      | 5,826.75  |
| 30%      | 5,881.46  |
| 40%      | 5,960.24  |
| 50%      | 6,141.64  |
| 60%      | 6,336.07  |
| 70%      | 6,543.36  |
| 80%      | 6,756.10  |

| Hit<br>90%<br>Rate | M ops/sec |
|--------------------|-----------|
| 100%               | 7,318.36  |

**Key Observation:** Lookup performance is extremely high (5.8-7.3 billion ops/sec). Higher hit rates improve performance due to early termination when a key is found.

Experiment 3: Load Factor Impact

**2 Hash Functions:** All load factors FAILED (even at 2.0x table size)

**3 Hash Functions:**

| Load Factor | M ops/sec | Status    |
|-------------|-----------|-----------|
| 1.01        | 0.00      | FAILED    |
| 1.02        | 0.00      | FAILED    |
| 1.05        | 0.00      | FAILED    |
| 1.10        | 0.00      | FAILED    |
| 1.20        | 0.00      | FAILED    |
| 1.30        | 4,019.61  | OK        |
| 1.40        | 4,581.96  | OK (Best) |
| 1.50        | 4,231.81  | OK        |
| 1.60        | 3,341.75  | OK        |
| 1.70        | 2,980.76  | OK        |
| 1.80        | 2,709.74  | OK        |
| 1.90        | 2,540.06  | OK        |
| 2.00        | 2,396.11  | OK        |

**Key Observation:** Optimal load factor is **1.4** (table size = 1.4n) with peak throughput of 4,582 M ops/sec. Load factors below 1.3 fail due to high collision probability.

Experiment 4: Eviction Chain Length (2^24 keys, 1.4n table)

**2 Hash Functions:** All eviction chain lengths triggered REHASH

**3 Hash Functions:**

| MaxEvict | Multiplier | M ops/sec | Status  |
|----------|------------|-----------|---------|
| 24       | 1×log(n)   | 0.00      | REHASH  |
| 48       | 2×log(n)   | 0.00      | REHASH  |
| 72       | 3×log(n)   | 785.37    | SUCCESS |
| 96       | 4×log(n)   | 785.29    | SUCCESS |
| 120      | 5×log(n)   | 785.35    | SUCCESS |
| 144      | 6×log(n)   | 785.20    | SUCCESS |
| 192      | 8×log(n)   | 784.64    | SUCCESS |
| 240      | 10×log(n)  | 784.58    | SUCCESS |

**Key Observation:** Minimum eviction chain length for success is **3×log(n) = 72**. Performance is consistent (~785 M ops/sec) for all successful configurations, so **3×log(n)** is optimal as it minimizes wasted work.

## 7. Conclusions

---

1. **Hash Functions: 3 hash functions are essential** for reliable operation. With 2 hash functions, insertion fails at  $2^{24}$  keys even with 50% load factor, while 3 hash functions succeed with high throughput. The extra hash function provides more alternative positions for evicted keys, dramatically reducing cycle probability.
2. **Optimal Load Factor:** Recommended table size is **1.4n** (load factor 1.4), achieving peak insertion throughput of 4,582 M ops/sec. Load factors below 1.3 fail due to insufficient slack for eviction chains.
3. **Eviction Chain Length:** Optimal maxEvict value is  **$3 \times \log(n)$**  (72 for  $n=2^{24}$ ). Values of  $1-2 \times \log(n)$  are insufficient and cause rehashing. Values above  $3 \times \log(n)$  provide no performance benefit.
4. **Throughput:**
  - Peak insertion rate: **1,166 M ops/sec** (2 hash,  $2^{23}$  keys) and **918 M ops/sec** (3 hash,  $2^{23}$  keys)
  - Peak lookup rate: **7,318 M ops/sec** (100% hit rate)
  - The V100's high memory bandwidth (900 GB/s) enables excellent hash table performance
5. **Lookup Performance:** Interestingly, higher hit rates improve lookup throughput (6,596 to 7,318 M ops/sec). This is because successful lookups terminate early after finding the key, while misses must check all k hash positions.

## Appendix: Source Files

---

- `Makefile` - Build configuration
- `run_experiments.sh` - Script to run all experiments
- `src/*.cu`, `src/*.cuh` - Source code